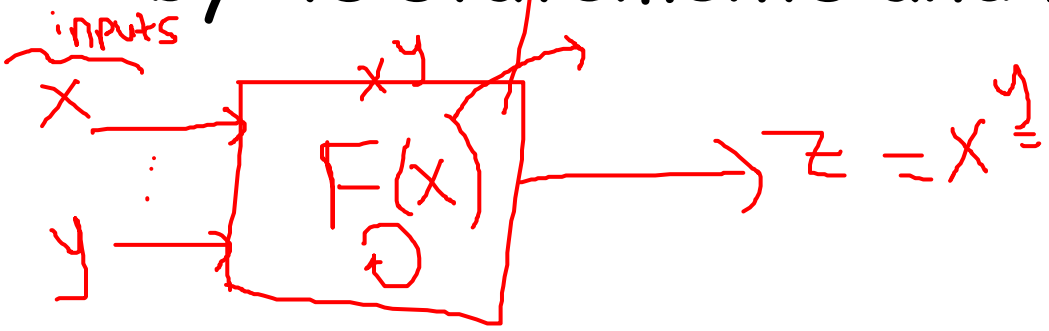
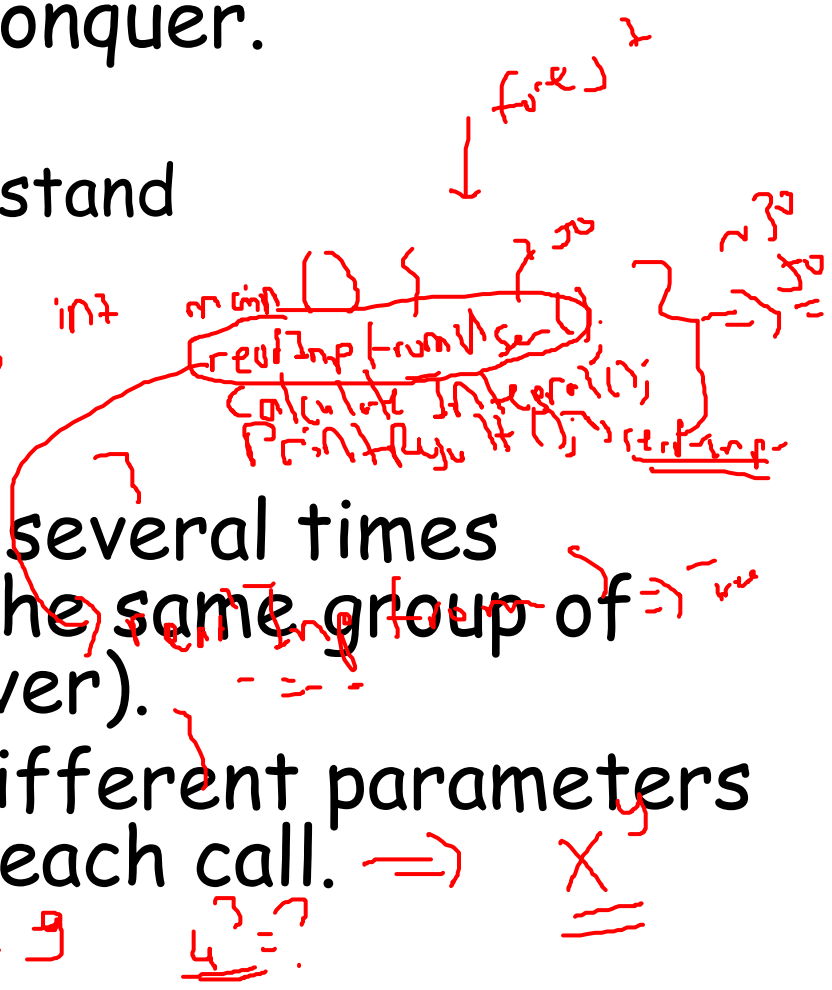
Functions

- A function groups a set of related statements under a single title.
- You can "call" the function using its name.
- You may also provide parameters to the function during the call.
- A function performs the actions defined by its statements and returns a result.



Motivation

1. Programming-in-the-large is feasible using functions: Divide and conquer.
   - Easier to develop
   - Easier to read and understand
   - Easier to maintain
   - Easier to reuse
   - Provides abstraction
2. You can call a function several times (instead of repeating the same group of statements over and over).
3. You may also provide different parameters to the function during each call.



Functions

- Syntax:

```
return_type function_name (parameter_list)
{   local_variable_definitions
    statement(s)
}
```

- Functions operate on their parameters and produce results.
  - The result is returned via the return value.
- The parameters may or may not be altered by the function.
- If return_type is missing, it means int.

Example

Consider the polynomial

$$P(x) = 8x^5 + 5x^4 + 6x^3 + 3x^2 + 4x + 2$$

Read a value for x and display the result.

Example: Solution without functions

```
#include <stdio.h>
int main()
{
   float P_x, x, t;
   int i;

   P_x=0;
   scanf("%f", &x);
   /* Calculate term x^5 */
   for (t=1, i=0; i<5; i++)
      t *= x;
   P_x += 8*t;
   /* Calculate term x^4 */
   for (t=1, i=0; i<4; i++)
      t *= x;
   P_x += 5*t;
   /* Calculate term x^3 */
   for (t=1, i=0; i<3; i++)
      t *= x;
   P_x += 6*t;
   /* Calculate term x^2 */
   for (t=1, i=0; i<2; i++)
      t *= x;
   P_x += 3*t;
   /* Calculate term x^1 and x^0 */
   P_x += 4*x+2;
   printf("Result=%f\n", P_x);
   return 0;
}
```

Example: Solution with functions

```
#include <stdio.h>

/* Calculate a^b */
float power(float a, int b)
{
   float result=1;

   for (; b>0; b--)
      result *= a;
   return result;
}

int main()
{
   float P_x, x;

   scanf("%f", &x);
   P_x = 8 * power(x,5) +
         5 * power(x,4) +
         6 * power(x,3) +
         3 * power(x,2) +
         4 * x + 2;
   printf("Result=%f\n", P_x);
   return 0;
}
```

We will visit this example again.

Functions

- A function that is being called is named callee.
- The function from which the call is made is named caller.
- In the previous slide, `main()` is the caller, `power()` is the callee.

Functions

- We will cover functions in the following order:
  - Void functions
  - Functions without parameters
  - Functions with parameters
  - Functions that return a value
  - Functions that alter their parameters

Void functions

- A function that is not supposed to return a value has a return type of **`void`**.
  - Note that skipping the return type makes it an integer function, not a void function.
- If callee has nothing to send back to the caller, make callee a void function.

Example: Void functions

```
#include <stdio.h>

void isosceles_triangle()
{  int line, i, j;

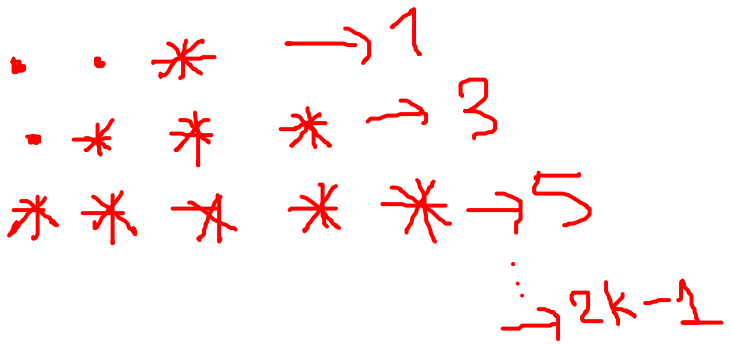
   scanf("%d", &line);
   for (i = 1; i<=line; i++)
   {
      for (j = 0; j<line - i; j++)
         printf(" ");
      for (j = 0; j<i*2-1; j++)
         printf("*");
      printf("\n");
   }
}

int main()
{  int i, n;

   scanf("%d", &n);
   for (i=0; i<n; i++)
      isosceles_triangle();
   return 0;
}
```

Note that **`isosceles_triangle()`** makes I/O, but it does not take any parameters or return any value.

call function multiple times



Functions with parameters

- You can direct how a function operates "within the program" if you use parameters.

Example: power() function without parameters

```
float power()
{
    float result=1, a ; int b;

    scanf("%f %d",&a,&b);
    for (; b>0; b--)
       result *= a;
    return result;
}
```

- `power()` finds `result` based on the values read from the input. You cannot specify `a` and `b` from the `main()` function.

Example: power() function with parameters

```
float power(float a, int b)
{
    float result=1;

    for (; b>0; b--)
        result *= a;
    return result;
}
```

- Now, `power()` finds `result` based on the parameters specified by the caller. So, you can direct it within the program

Changes in parameters are not reflected

- Note that in the previous example, the changes made on the parameters are not reflected to the caller.
- This rule applies as long as you use value parameters.

Changes in parameters are not reflected

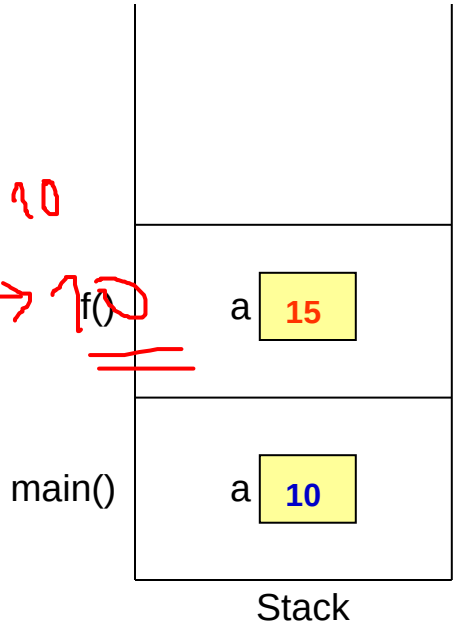
```
#include <stdio.h>

void f(int a)
{
   a+=5;
   printf("in function f(): a=%d\n", a);
}

int main()
{  int a=10;
   printf("in main(), before calling f(): a=%d\n",a);
   f(a);
   printf("in main(), after calling f(): a=%d\n",a);
}
```

OUTPUT

```
in main(), before calling f(): a=10
in function f(): a=15
in main(), after calling f(): a=10
```



Return value

- A function performs a task and finds a result.
- The result replaces the function call.
- Eg: For the function call below

```
y = 3*power(2,5)+4;
```

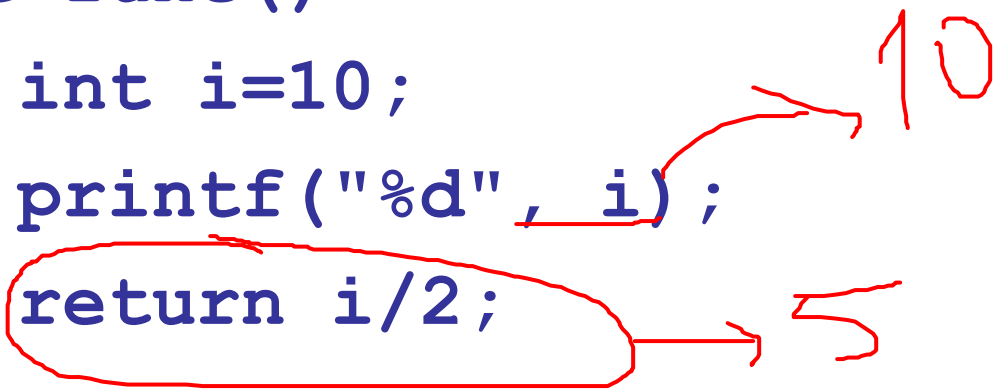
the function `power()` calculates 2^5 and the function call is replaced by `32`. So the statement becomes

```
y = 3*32+4;
```

Return value

- Note that the return value has got nothing to do with input/output.
- Eg:

```
int func()
{   int i=10;
    printf("%d", i);
    return i/2;
}
```



- `func()` outputs `10`, but returns `5`.

Example: Functions with return type and parameters

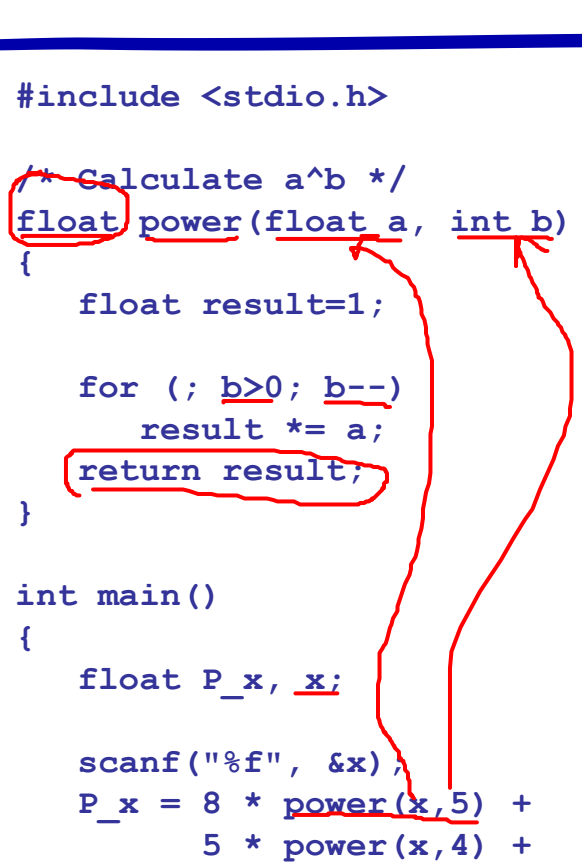
```
#include <stdio.h>

/* Calculate a^b */
float power(float a, int b)
{
   float result=1;

   for (; b>0; b--)
      result *= a;
   return result;
}

int main()
{
   float P_x, x;

   scanf("%f", &x);
   P_x = 8 * power(x,5) +
         5 * power(x,4) +
         6 * power(x,3) +
         3 * power(x,2) +
         4 * x + 2;
   printf("Result=%f\n", P_x);
   return 0;
}
```



Example: Functions with return type and parameters

- What is the output of the following program?

```
#include <stdio.h>
float half(int a)
{
    return a/2;
}
int main()
{   float r;
    r=half(10)/3;
    printf("Result=%f", r);
}
```